

Practica 2

Describir y simular una ALU combinacional.

Operaciones requeridas sobre operandos A y B de 8 bits.

Suma $A+B$ (con bandera de carry out)

Resta $A-B$ (Con bandera de borrow en carry out)

AND bit a bit

OR bit a bit

XOR bit a bit

NOT A bit a bit

bitShiftA lógico izquierdo por 1

bitShiftA lógico derecho por 1

bitFlags a producir: zero, acarry , sign

Señales de control: opcode de 3 bits que selecciona operación.

```

1  // Main ALU module
2  module ALU_M(A,B,sel,Z,zero,carry,sign);
3
4  input [7:0] A,B; // 8-bit inputs
5  input [2:0] sel; // 3-bit select line
6  output [7:0] Z; // 8-bit output
7  output zero,carry,sign; // bitFlags
8
9  wire [7:0] AoS,AND,OR,XOR,NOT,LLS,LRS; // Intermediate wires for each operation
10 wire [8:0] TC; // Output from subtraction
11 wire CoB, Enable_S;
12
13 // Instantiate each operation module
14 ADD_8b U0(.A(A),.B(TC),.Cin(Enable_S),.S(AoS),.Cout(CoB));
15 TC_8b U1(.A(B),.Enable(Enable_S),.Z(TC));
16 AND_8b U2(.A(A),.B(B),.Z(AND));
17 OR_8b U3(.A(A),.B(B),.Z(OR));
18 XOR_8b U4(.A(A),.B(B),.Z(XOR));
19 NOT_8b U5(.A(A),.Z(NOT));
20 LLS_8b U6(.A(A),.Z(LLS));
21 LRS_8b U7(.A(A),.Z(LRS));
22
23 assign Enable_S = (~sel[2]) & (~sel[1]) & (sel[0]); // Enable signal for addition/subtraction
24 assign carry = ((~sel[2]) & (~sel[1]) & (~sel[0])) & CoB; // Carry flag only for addition
25 assign sign = ((~sel[2]) & (~sel[1]) & (sel[0])) & CoB;
26 assign zero = ~(|Z); // Zero flag
27
28 assign Z = (sel == 3'b000) ? AoS :
29           (sel == 3'b001) ? AoS : // Subtraction uses the same output as addition
30           (sel == 3'b010) ? AND :
31           (sel == 3'b011) ? OR :
32           (sel == 3'b100) ? XOR :
33           (sel == 3'b101) ? NOT :
34           (sel == 3'b110) ? LLS :
35           (sel == 3'b111) ? LRS : 8'b00000000; // Default case
36 endmodule
37
38 //8-bit Adder
39 module ADD_8b(A,B,Cin,S,Cout);
40 input [7:0] A;
41 input [8:0] B;
42 input Cin;
43 output [7:0] S;
44 output Cout;
45 wire [7:0] CO_S;
46 wire dummy;
47
48 FA U0(.A(A[0]),.B(B[0]),.Cin(Cin),.S(S[0]),.Cout(CO_S[0]));
49 FA U1(.A(A[1]),.B(B[1]),.Cin(CO_S[0]),.S(S[1]),.Cout(CO_S[1]));
50 FA U2(.A(A[2]),.B(B[2]),.Cin(CO_S[1]),.S(S[2]),.Cout(CO_S[2]));
51 FA U3(.A(A[3]),.B(B[3]),.Cin(CO_S[2]),.S(S[3]),.Cout(CO_S[3]));
52 FA U4(.A(A[4]),.B(B[4]),.Cin(CO_S[3]),.S(S[4]),.Cout(CO_S[4]));
53 FA U5(.A(A[5]),.B(B[5]),.Cin(CO_S[4]),.S(S[5]),.Cout(CO_S[5]));
54 FA U6(.A(A[6]),.B(B[6]),.Cin(CO_S[5]),.S(S[6]),.Cout(CO_S[6]));
55 FA U7(.A(A[7]),.B(B[7]),.Cin(CO_S[6]),.S(S[7]),.Cout(CO_S[7]));
56 FA U8(.A(1'b0),.B(B[8]),.Cin(CO_S[7]),.S(Cout),.Cout(dummy));
57 endmodule

```

```

59 //8-bit Full Adder
60 module FA (A,B,S,Cin,Cout);
61 input A,B,Cin;
62 output S,Cout;
63
64 assign S = (A ^ B) ^ Cin;
65 assign Cout = (A & B) | ((A ^ B) & Cin);
66 endmodule
67
68 //8-bit twos complement
69 module TC_8b(A,Enable,Z);
70 input [7:0] A;
71 input Enable;
72 output [8:0] Z;
73 wire [8:0] A_1; //Add the sign bit
74
75 assign A_1[8] = 1'b0;
76 assign A_1[7:0] = A;
77 assign Z = A_1 ^ {9{Enable}}; // If Enable is 1, invert bits
78 endmodule
79
80 //8-bit AND
81 module AND_8b(A,B,Z);
82 input [7:0] A,B;
83 output [7:0] Z;
84
85 assign Z = A & B;
86 endmodule
87
88 //8-bit OR
89 module OR_8b(A,B,Z);
90 input [7:0] A,B;
91 output [7:0] Z;
92
93 assign Z = A | B;
94 endmodule
95
96 //8-bit XOR
97 module XOR_8b(A,B,Z);
98 input [7:0] A,B;
99 output [7:0] Z;
100
101 assign Z = A ^ B;
102 endmodule
103
104 //8-bit NOT
105 module NOT_8b(A,Z);
106 input [7:0] A;
107 output [7:0] Z;
108
109 assign Z = ~A;
110 endmodule
111
112 //8-bit Logical Left Shift
113 module LLS_8b(A,Z);
114 input [7:0] A;
115 output [7:0] Z;

```

```
116
117   assign Z = A << 1;
118   endmodule
119
120   //8-bit Logical Right Shift
121   module LRS_8b(A,Z);
122     input [7:0] A;
123     output [7:0] Z;
124
125     assign Z = A >> 1;
126   endmodule
```

```

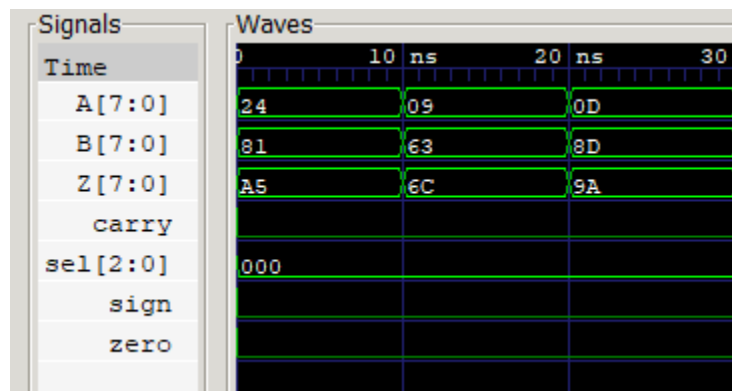
1  //ALU_M testbench
2  `timescale 1ns/1ps
3  `include "ALU_M.v" //Import the file to which we are making its testbench
4
5  module ALU_M_tb ();
6  integer i; //Variable for counting
7  reg [7:0] A,B; //8-bit inputs
8  reg [2:0] sel; //3-bit select input
9  wire [7:0] Z; //8-bit output
10 wire zero,carry,sign; //bitFlags
11
12 //Instantiate the module under test
13 ALU_M uut (A,B,sel,Z,zero,carry,sign);
14
15 initial begin
16
17     $dumpfile("ALU_M_tb.vcd"); //Create the file that will give us the waveform of our output
18     $dumpvars(0,ALU_M_tb); //Record all variable changes in the testbench
19
20
21     sel = 3'b000; // Addition
22     for (i = 0; i < 3; i = i + 1) begin
23         A = $random & 8'hFF;
24         B = $random & 8'hFF;
25         #10;
26     end
27
28     sel = 3'b001; // Addition
29     for (i = 0; i < 3; i = i + 1) begin
30         A = $random & 8'hFF;
31         B = $random & 8'hFF;
32         #10;
33     end
34
35     sel = 3'b010; // Addition
36     for (i = 0; i < 3; i = i + 1) begin
37         A = $random & 8'hFF;
38         B = $random & 8'hFF;
39         #10;
40     end
41
42     sel = 3'b011; // Addition
43     for (i = 0; i < 3; i = i + 1) begin
44         A = $random & 8'hFF;
45         B = $random & 8'hFF;
46         #10;
47     end
48
49     sel = 3'b100; // Addition
50     for (i = 0; i < 3; i = i + 1) begin
51         A = $random & 8'hFF;
52         B = $random & 8'hFF;
53         #10;
54     end
55

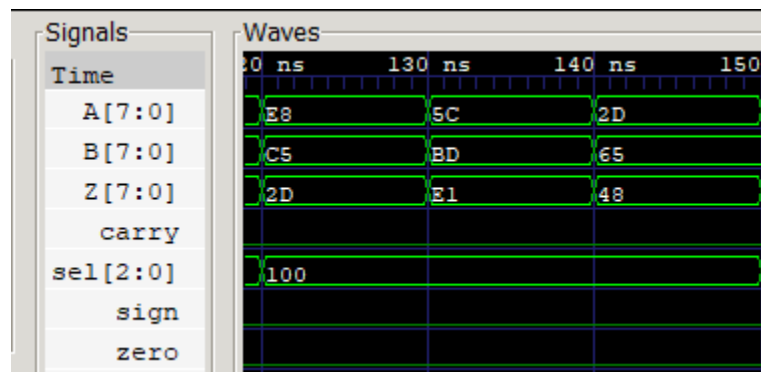
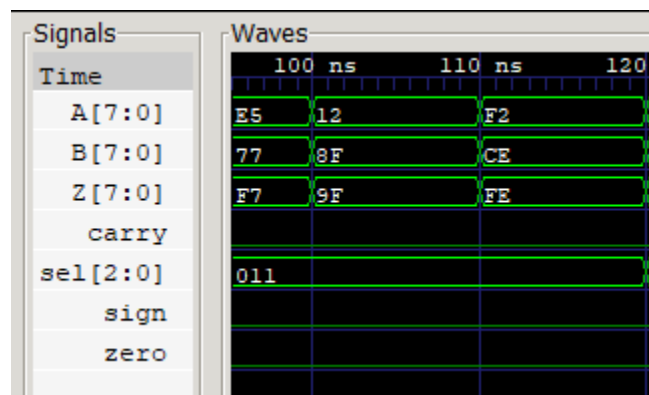
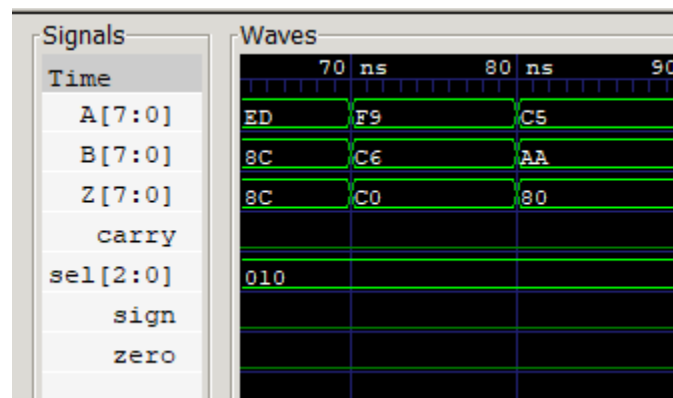
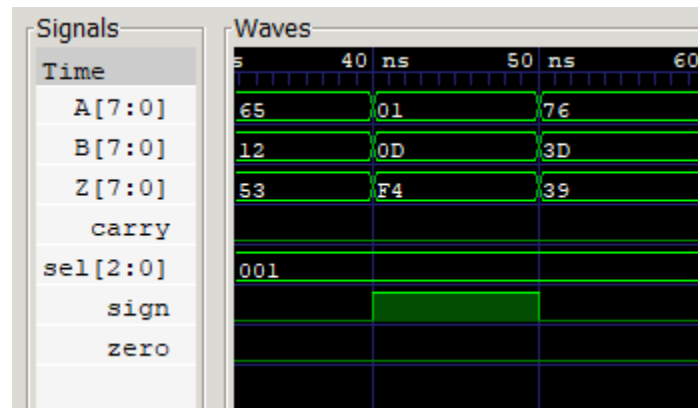
```

```

55
56     sel = 3'b101; // Addition
57     for (i = 0; i < 3; i = i + 1) begin
58         A = $random & 8'hFF;
59         #10;
60     end
61
62     sel = 3'b110; // Addition
63     for (i = 0; i < 3; i = i + 1) begin
64         A = $random & 8'hFF;
65         #10;
66     end
67
68     sel = 3'b111; // Addition
69     for (i = 0; i < 3; i = i + 1) begin
70         A = $random & 8'hFF;
71         #10;
72     end
73
74     //Especial cases
75     //Addition with carry
76     A = 8'hFF; B = 8'h01; sel = 3'b000; #10;
77     //Subtraction with borrow
78     A = 8'h00; B = 8'h01; sel = 3'b001; #10;
79     //Subtraction resulting in zero
80     A = 8'hFF; B = 8'hFF; sel = 3'b001; #10;
81     //AND resulting in zero
82     A = 8'hF0; B = 8'h0F; sel = 3'b010; #10;
83     //OR resulting in zero
84     A = 8'h00; B = 8'h00; sel = 3'b011; #10;
85     //XOR resulting in zero
86     A = 8'hAA; B = 8'hAA; sel = 3'b100; #10;
87     //NOT operation on zero
88     A = 8'hFF; sel = 3'b101; #10;
89     //Left logical shift resulting in overflow
90     A = 8'h80; sel = 3'b110; #10;
91     //Right logical shift resulting in overflow
92     A = 8'h01; sel = 3'b111; #10;
93
94     $display("Test completed"); //Show message in console when test is completed
95
96 end
97
98 endmodule
99

```





Código

```
// Main ALU module
module ALU_M(A,B,sel,Z,zero,carry,sign);

input [7:0] A,B; // 8-bit inputs
input [2:0] sel; // 3-bit select line
output [7:0] Z; // 8-bit output
output zero,carry,sign; // bitFlags

wire [7:0] AoS,AND,OR,XOR,NOT,LLS,LRS; // Intermediate wires for each operation
wire [8:0] TC; // Output from subtraction
wire CoB, Enable_S;

// Instantiate each operation module
ADD_8b U0(.A(A),.B(TC),.Cin(Enable_S),.S(AoS),.Cout(CoB));
TC_8b U1(.A(B),.Enable(Enable_S),.Z(TC));
AND_8b U2(.A(A),.B(B),.Z(AND));
OR_8b U3(.A(A),.B(B),.Z(OR));
XOR_8b U4(.A(A),.B(B),.Z(XOR));
NOT_8b U5(.A(A),.Z(NOT));
LLS_8b U6(.A(A),.Z(LLS));
LRS_8b U7(.A(A),.Z(LRS));

assign Enable_S = (~sel[2]) & (~sel[1]) & (sel[0]); // Enable signal for addition/subtraction
assign carry = ((~sel[2]) & (~sel[1]) & (~sel[0])) & CoB; // Carry flag only for addition
assign sign = ((~sel[2]) & (~sel[1]) & (sel[0])) & CoB;
assign zero = ~(|Z); // Zero flag

assign Z = (sel == 3'b000) ? AoS :
    (sel == 3'b001) ? AoS : // Subtraction uses the same output as addition
    (sel == 3'b010) ? AND :
    (sel == 3'b011) ? OR :
    (sel == 3'b100) ? XOR :
    (sel == 3'b101) ? NOT :
    (sel == 3'b110) ? LLS :
    (sel == 3'b111) ? LRS : 8'b00000000; // Default case
endmodule

//8-bit Adder
module ADD_8b(A,B,Cin,S,Cout);
input [7:0] A;
input [8:0] B;
input Cin;
output [7:0] S;
```

```

output Cout;
wire [7:0] CO_S;
wire dummy;

FA U0(.A(A[0]),.B(B[0]),.Cin(Cin) ,.S(S[0]),.Cout(CO_S[0]));
FA U1(.A(A[1]),.B(B[1]),.Cin(CO_S[0]),.S(S[1]),.Cout(CO_S[1]));
FA U2(.A(A[2]),.B(B[2]),.Cin(CO_S[1]),.S(S[2]),.Cout(CO_S[2]));
FA U3(.A(A[3]),.B(B[3]),.Cin(CO_S[2]),.S(S[3]),.Cout(CO_S[3]));
FA U4(.A(A[4]),.B(B[4]),.Cin(CO_S[3]),.S(S[4]),.Cout(CO_S[4]));
FA U5(.A(A[5]),.B(B[5]),.Cin(CO_S[4]),.S(S[5]),.Cout(CO_S[5]));
FA U6(.A(A[6]),.B(B[6]),.Cin(CO_S[5]),.S(S[6]),.Cout(CO_S[6]));
FA U7(.A(A[7]),.B(B[7]),.Cin(CO_S[6]),.S(S[7]),.Cout(CO_S[7]));
FA U8(.A(1'b0),.B(B[8]),.Cin(CO_S[7]),.S(Cout),.Cout(dummy));
endmodule

```

```

//8-bit Full Adder
module FA (A,B,S,Cin,Cout);
input A,B,Cin;
output S,Cout;

assign S = (A ^ B) ^ Cin;
assign Cout = (A & B) | ((A ^ B) & Cin);
endmodule

```

```

//8-bit twos complement
module TC_8b(A,Enable,Z);
input [7:0] A;
input Enable;
output [8:0] Z;
wire [8:0] A_1; //Add the sign bit

assign A_1[8] = 1'b0;
assign A_1[7:0] = A;
assign Z = A_1 ^ {9{Enable}}; // If Enable is 1, invert bits
endmodule

```

```

//8-bit AND
module AND_8b(A,B,Z);
input [7:0] A,B;
output [7:0] Z;

assign Z = A & B;
endmodule

```

```
//8-bit OR
module OR_8b(A,B,Z);
input [7:0] A,B;
output [7:0] Z;
```

```
assign Z = A | B;
endmodule
```

```
//8-bit XOR
module XOR_8b(A,B,Z);
input [7:0] A,B;
output [7:0] Z;
```

```
assign Z = A ^ B;
endmodule
```

```
//8-bit NOT
module NOT_8b(A,Z);
input [7:0] A;
output [7:0] Z;
```

```
assign Z = ~A;
endmodule
```

```
//8-bit Logical Left Shift
module LLS_8b(A,Z);
input [7:0] A;
output [7:0] Z;
```

```
assign Z = A << 1;
endmodule
```

```
//8-bit Logical Right Shift
module LRS_8b(A,Z);
input [7:0] A;
output [7:0] Z;
```

```
assign Z = A >> 1;
endmodule
```

Testbench

```
//ALU_M testbench
`timescale 1ns/1ps
```

```

`include "ALU_M.v" //Import the file to which we are making its testbench

module ALU_M_tb ();
integer i; //Variable for counting
reg [7:0] A,B; //8-bit inputs
reg [2:0] sel; //3-bit select input
wire [7:0] Z; //8-bit output
wire zero,carry,sign; //bitFlags

//Instantiate the module under test
ALU_M uut (A,B,sel,Z,zero,carry,sign);

initial begin

    $dumpfile("ALU_M_tb.vcd"); //Create the file that will give us the waveform of our output
    $dumpvars(0,ALU_M_tb); //Record all variable changes in the testbench

    sel = 3'b000; // Addition
    for (i = 0; i < 3; i = i + 1) begin
        A = $random & 8'hFF;
        B = $random & 8'hFF;
        #10;
    end

    sel = 3'b001; // Addition
    for (i = 0; i < 3; i = i + 1) begin
        A = $random & 8'hFF;
        B = $random & 8'hFF;
        #10;
    end

    sel = 3'b010; // Addition
    for (i = 0; i < 3; i = i + 1) begin
        A = $random & 8'hFF;
        B = $random & 8'hFF;
        #10;
    end

    sel = 3'b011; // Addition
    for (i = 0; i < 3; i = i + 1) begin
        A = $random & 8'hFF;
        B = $random & 8'hFF;
        #10;
    end

```

```

end

sel = 3'b100; // Addition
  for (i = 0; i < 3; i = i + 1) begin
    A = $random & 8'hFF;
    B = $random & 8'hFF;
    #10;
  end

sel = 3'b101; // Addition
  for (i = 0; i < 3; i = i + 1) begin
    A = $random & 8'hFF;
    #10;
  end

sel = 3'b110; // Addition
  for (i = 0; i < 3; i = i + 1) begin
    A = $random & 8'hFF;
    #10;
  end

sel = 3'b111; // Addition
  for (i = 0; i < 3; i = i + 1) begin
    A = $random & 8'hFF;
    #10;
  end

//Especial cases
//Addition with carry
A = 8'hFF; B = 8'h01; sel = 3'b000; #10;
//Subtraction with borrow
A = 8'h00; B = 8'h01; sel = 3'b001; #10;
//Subtraction resulting in zero
A = 8'hFF; B = 8'hFF; sel = 3'b001; #10;
//AND resulting in zero
A = 8'hF0; B = 8'h0F; sel = 3'b010; #10;
//OR resulting in zero
A = 8'h00; B = 8'h00; sel = 3'b011; #10;
//XOR resulting in zero
A = 8'hAA; B = 8'hAA; sel = 3'b100; #10;
//NOT operation on zero
A = 8'hFF; sel = 3'b101; #10;
//Left logical shift resulting in overflow
A = 8'h80; sel = 3'b110; #10;

```

```
//Right logical shift resulting in overflow  
A = 8'h01; sel = 3'b111; #10;  
  
$display("Test completed"); //Show message in console when test is completed  
  
end  
  
endmodule
```